

Le paradigme impératif

concepts généraux et exemples de mise en œuvre dans différents
langages de programmation

Table des matières

I. Fondements de l'informatique.....	3
I.1. Les 4 piliers de l'informatique.....	3
I.2. Les interfaces.....	3
II. Paradigme impératif.....	4
II.1. Préambule :.....	4
II.2. Vocabulaire abordé.....	4
III. Typage.....	6
III.1. Concepts généraux.....	6
III.2. Typage fort vs typage faible.....	6
III.3. Types de base courants.....	7
III.4. Extension des types.....	8
IV. Variables.....	10
IV.1. Variable et affectation (rappels).....	10
IV.2. Stockage par valeur, stockage par référence.....	11
IV.3. Signification d'un nom de variable.....	11
IV.4. Exemples d'utilisations de variables.....	12
IV.5. Notion d'égalité.....	14
V. Les conditionnelles.....	15
V.1. Le test à 2 branches.....	15
V.2. Le test à plus de 2 branches.....	15
V.3. Le test à valeur d'expression.....	18
VI. Les boucles.....	19
VI.1. Boucles bornées.....	19
VI.2. Boucles non bornées.....	21
VI.3. On a fait le tour ?.....	21
VII. Les sauts.....	22
VII.1. saut « sans retour ».....	22
VII.2. Saut « avec retour ».....	22
VII.3. Remarque générale concernant les sauts.....	23
VII.4. Saut spécifiques aux boucles.....	23
VIII. Les fonctions.....	24
VIII.1. Éviter la redondance de blocs d'instructions.....	24
VIII.2. Éviter la redondance d'expressions.....	26
VIII.3. Différences procédures/fonctions.....	29
VIII.4. les méthodes.....	29

IX. Interprétation vs Compilation.....	32
IX.1. Exécution d'un programme.....	32
IX.2. Exemple de mise en œuvre avec un programme de type « Hello World ».....	32
IX.3. Interprétation vs Exécution : avantages et inconvénients.....	33
IX.4. Autres situations.....	35
IX.5. Facilités de mise en œuvre.....	35
X. Niveau de langage.....	37

I. Fondements de l'informatique

I.1. Les 4 piliers de l'informatique

Gilles Dowek a défini ce qu'il appelle les 4 piliers de l'informatique :

- les données
- les algorithmes
- les langages
- les machines

Les **données** (ou informations) sont l'objet même de l'existence de l'informatique (étymologiquement, informatique vient de la contraction de « information » et de « automatique » ; il s'agit donc de pouvoir traiter automatiquement des informations).

Les **algorithmes** sont les procédés permettant de traiter les données.

Les **langages** sont les moyens de décrire des algorithmes. Un **programme** est la réification (ou implantation, implémentation, ...) d'un algorithme dans un langage.

Les **machines** (concrètes ou abstraites) sont les outils capables d'exécuter un programme écrit dans un langage donné. Un **processus** est un programme en cours d'exécution sur une machine.

I.2. Les interfaces

Aux 4 piliers de l'informatique, Gérard Berry rajoute la notion d'**interface** : la façon dont un programme échange avec le monde extérieur, en entrée (récupération d'informations : clavier, capteur de température, souris, fichier, réseau, etc.) ou en sortie (écran, imprimante, fichier, réseau, etc.) : un programme qui fonctionne sans échange avec l'extérieur est fonctionnellement comparable à un cube en bois.

II. Paradigme impératif

II.1. Préambule :

Ce document résume quelques unes des notions présentées. Dans la suite, de ce document, on mélange, provisoirement, les notions de « programme » et « programme en cours d'exécution ».

II.2. Vocabulaire abordé

II.2.1. Langage impératif, instruction programme

Un **langage impératif** est un langage dont les **programmes** consistent principalement en une séquence d'instructions.

Une **instruction** est un ordre donné à l'ordinateur. Cet instruction se traduit par un changement d'état et éventuellement un effet de bord (voir section II.2.2).

Un **bloc d'instructions** est une suite d'instructions pouvant être considérée comme une seule instruction. Suivant les langages, cette structure de bloc peut prendre diverses formes : accolades en C, Java, Perl, ..., crochets en Logo, indentation particulière en Python, BEGIN ... END en Pascal, etc.

Un langage est défini par sa **grammaire** (ce qui est correct sur le plan de la forme), sa **sémantique** (ce que fait une instruction donnée) et son **vocabulaire** (la liste des mots autorisés).

Un langage permet en général de manipuler des valeurs soit directement, soit par un calcul. On appelle la partie du langage permettant de définir une valeur une **expression**. Voici quelques exemples d'expression :

- 12
- $3 * 4.5$
- "Bonjour"

Ne pas confondre *instruction* et *expression*

Parmi les instructions, on trouve notamment l'affectation, qui permet d'associer une valeur à une variable, et l'affichage.

Lorsqu'un programme est en cours d'exécution, il existe la notion d'« **instruction courante** », c'est-à-dire d'instruction en cours d'exécution.

II.2.2. Variable, affectation, état

Une **variable** est une zone de stockage (abstraite ou concrète) nommée permettant de stocker une valeur.

L'**affectation** est l'opération qui consiste à associer une information à une variable. La notion d'affectation est une notion que l'on trouve dans tous les langages impératifs. Le symbole peut varier d'un langage à l'autre (signe « = » en Python, Java ou C par exemple, signe « := » en Pascal, etc.).

On appelle **état** d'un programme l'ensemble des valeurs de ses variables à un instant donné (éventuellement augmenté de l'instruction courante).

Une instruction permet de passer d'un état e_1 à un état e_2 . Certaines instructions ne modifient pas la valeur de l'état (sauf si l'on inclut la référence à l'instruction courante dans l'état), mais présentent des **effets de bord**, c'est-à-dire des modifications de l'environnement du programme (affichage sur un écran, écriture dans un fichier, envoi d'informations sur un réseau...).

On appelle **trace d'exécution** la succession des états du programme au cours de son exécution.

Tester différents langages en ligne :

Python : <https://pythontutor.com/>

Java : <https://pythontutor.com/>

C : <https://pythontutor.com/>

Pascal : https://www.onlinegdb.com/online_pascal_compiler

Basic : <https://www.calormen.com/jsbasic/>

Logo : <https://www.calormen.com/jslogo/>

III. Typage

III.1. Concepts généraux

Les expressions ont toujours un type. En première approche, un **type** peut être vu comme un ensemble de valeurs. La notion de type est fondamentale en informatique. Elle définit la façon dont les données sont représentées, et les opérations qu'on peut leur appliquer.

Une expression peut être une valeur, une variable (évaluée à sa valeur), ou encore l'application d'un opérateur ou d'une fonction sur des expressions plus simples.

Chaque langage définit en général ses propres types de base. Cela peut varier d'un langage à un autre. Souvent, on trouve les notions suivantes :

- nombres
 - entiers
 - flottants
- textes
 - caractères
 - chaînes de caractères
- booléens

D'un langage à un autre, les spécificités des types utilisés pour chacune de ces notions peuvent varier.

III.2. Typage fort vs typage faible

Selon les langages, des **conversions implicites** entre types peuvent être proposées. On parlera plutôt de **typage faible** pour un langage faisant beaucoup de conversions automatiques, et plutôt de **typage fort** pour un langage en faisant peu.

Suivant les langages, les variables peuvent être typées ou non. Quand une variable est typée, elle ne peut contenir que des valeurs de même type que celui de la variable. Le fait que les variables soient typées contribue à renforcer la « force » du typage.

Un même opérateur peut s'appliquer sur des données de types différents. On parle alors de **surcharge d'opérateur** / **opérateur surchargé**.

En résumé, ce qui va dans le sens d'un typage faible :

- des conversions implicites
- des variables non typées
- des opérateurs surchargés

Un **typage faible** permet souvent d'écrire des programmes plus rapidement et plus simplement.

Un **typage fort** permet d'écrire des programmes plus sûrs.

III.3. Types de base courants

III.3.1. Nombres entiers

Suivant les langages, on peut avoir différents types pour les entiers, selon l'intervalle d'entiers concernés :

- entiers sur 1 octet : $2^8 = 256$ valeurs différentes
- entiers sur 2 octets : $2^{16} = 65\,536$ valeurs différentes
- entiers sur 4 octets : 2^{32} valeurs différentes
- entiers sur 8 octets : 2^{64} valeurs différentes
- entiers sur un nombre indéterminé d'octets : nombres aussi grands que l'autorise la mémoire allouée au programmes

Exemple en Java :

- entiers sur 1 octet : type `byte`
- entiers sur 2 octets : type `short`
- entiers sur 4 octets : type `int`
- entiers sur 8 octets : type `long`
- entiers de longueur arbitraire : type `BigInteger`

III.3.2. Nombres « réels »

Les ordinateurs travaillent en base 2. Aussi, les nombres « réels » sur lesquels ils travaillent sont en général plutôt des nombres dyadiques, c'est à dire des nombres s'écrivant en base 2 avec un nombre de chiffres fini après la virgule. Il n'y a pas de bijection entre les nombres décimaux et les nombres dyadiques, ce qui peut impliquer des comportements semblant bizarres :

```
>>>2.7 + 1.2
3.9000000000000004
```

III.3.3. Booléens

Le type booléen correspond à un type définissant 2 valeurs : Vrai et Faux. C'est un type utilisé notamment lorsque le programme doit effectuer des tests (comme pour l'instruction `if` par exemple).

Dans certains langages, le type booléen n'existe pas ; les entiers jouent alors le rôle de booléen : la valeur 0 est interprétée comme « Faux », les autres valeurs sont interprétées comme « Vrai ». C'est le cas dans le langage C par exemple.

Dans certains langages, le type booléen existe, mais des conversions implicites permettent d'utiliser éventuellement d'autres types. Par exemple, en Python, pour les « objets », certaines valeurs seront interprétées comme `False`, d'autres comme `True`.

Des valeurs booléennes peuvent par exemples être calculées à partir des opérateurs de comparaison (`==`, `<`, `>`, etc.) ou à partir d'opérateurs logiques permettant de combiner des valeurs booléennes entre elles (`and`, `or`, `not` par exemple).

III.3.4. Conversions implicites ou non : exemple de l'addition

Dans beaucoup de l'engage, l'opérateur « + » permet d'additionner des nombres quels qu'ils soient. Les 3 exemples constituent par exemple des expressions valides en Python, C ou Java :

```
2 + 3
2.5 + 3.2
2 + 3.4
```

Mais l'opérateur « + » permet aussi la concaténation de chaîne de caractères en Python et en Java. Ainsi, dans ces langages, l'expression :

```
"bon" + "jour"
```

est une expression valide dont la valeur est "bonjour". Mais ce n'est pas valide en C.

En Java, l'utilisation de l'opérateur « + » entre une chaîne de caractères et une donnée d'un autre type entraîne automatiquement la conversion de l'autre donnée en une chaîne de caractères afin de pouvoir effectuer la concaténation. Ainsi, l'expression :

```
"v" + 2
```

est valide en Java et a pour valeur "v2", mais elle n'est pas valide en Python (il faudrait passer par une concaténation explicite : "v" + str(2)).

En Perl, l'opérateur « + » correspond à l'addition uniquement (il existe un autre opérateur pour la concaténation), et sont utilisation déclenche automatiquement la conversion de ses 2 opérandes (si nécessaire) en nombres. Le programme perl suivant :

```
print("21" + 3)
print(2 + "trois")
```

affiche :

```
24
2
```

En effet, la chaîne « 21 » de la première ligne est convertie en le nombre 21, et la chaîne «trois» du deuxième exemple, n'étant pas un nombre en chiffre, est convertie en le nombre 0.

III.4. Extension des types

Beaucoup de langages permettent d'enrichir l'ensemble des types de base. Cela se fait notamment via les 2 moyens suivants :

- les **tableaux** : cela permet de définir un type comme une liste ordonnée de taille fixe de données d'un autre type.

Exemple :

```
int[] a = {1, 5, 7} ;
```

- les **enregistrements** : cela permet de définir un type comme un ensemble de caractéristiques nommées ayant chacune leur propre type.

Exemple :

```
typedef struct {  
  int x ;  
  int y ;  
  char[] nom ;  
} point ;
```

Dans les langages orientés objets, la notion d'enregistrement sera souvent remplacée par les notions de classe ou de prototype.

IV. Variables

IV.1. Variable et affectation (rappels)

IV.1.1. Définition

Une **variable** est un emplacement de stockage abstrait nommé, permettant de stocker une valeur.

IV.1.2. Déclaration

Suivant les langages, une variable doit être **déclarée** (avec un type) ou non. Déclarer une variable consiste à annoncer explicitement son existence, en général en lui associant un type.

Dans certains langages, les variables ont un type immuable (on parle alors de variable typée) qui peut être spécifié soit explicitement par le programme soit inféré. On parle alors de variable typée. Dans les autres langages, les variables n'ont pas de type et peuvent donc, au cours de leur exécution contenir des données de types différents.

Exemple 1 : typage explicite des variables en C ou en Java

```
int x ;  
x = 2 ;
```

Ici, on déclare que la variable x est de type entier ; elle ne pourra donc contenir que des entiers.

Exemple 2 : typage inféré en Java

```
var x = 2.3 ;
```

Java autorise dans certains cas un typage implicite. Ici, le type de x n'est pas précisé par le programmeur ; c'est le compilateur java qui détermine, à partir de l'expression à droite du signe « = », que la variable x est de type double. Elle ne pourra donc contenir que des données de type double.

Exemple 3 : variable non typée en Python

```
x = 2  
x = "toto"
```

La variable x n'a pas besoin d'être déclarée. Sa première utilisation à gauche d'une affectation suffit à la déclarer. Elle n'est pas typée, et peut donc contenir des valeurs de différents types au fur et à mesure de l'exécution du programme.

IV.1.3. Affectation

L'**affectation** est l'opération qui consiste à **stocker une copie** d'une information dans une variable. La notion d'affectation est une notion que l'on trouve dans tous les langages impératifs. Le symbole peut varier d'un langage à l'autre (signe « = » en Python, Java ou C par exemple, signe « := » en Pascal, etc.).

IV.2. Stockage par valeur, stockage par référence

La valeur que contient la variable peut être soit l'information qu'on lui associe (on parle alors de **stockage par valeur**) soit une référence vers l'information (on parle alors de **stockage par référence**).

Une **référence** est un moyen d'identifier de manière univoque l'information (il peut s'agir par exemple de son adresse dans la mémoire de l'ordinateur).

2 variables `va` et `vb` peuvent contenir la même référence, c'est-à-dire faire référence à la même donnée. Aussi, modifier la donnée stockée par référence dans `va` modifiera aussi la donnée stockée par référence dans `vb` (puisque'il s'agit de la même donnée...).

Suivant les langages, la façon dont est géré le stockage des données varie. Voici quelques exemples :

- en Python les données ne sont stockées que par référence
- en Java, suivant le type des données, elles sont stockées par valeur ou par référence
- en C, il est possible de choisir si une variable contient une donnée ou une référence à une donnée.

L'affectation `a = b` **recopie** dans `a` la valeur contenue dans `b`. S'il s'agit d'un stockage par valeur, c'est la valeur qui est recopiée dans `a`, s'il s'agit d'un stockage par référence, c'est la référence qui est copiée dans `a`.

Dans certains langages, il existe des données **immuables** ou **non mutables** : ce sont des données dont la valeur ne peut pas changer.

Pointeur ou référence ?

Dans certains langages comme le C, on ne parle pas de référence, mais de pointeur. Même s'il s'agit de notions proches (et on les confondra souvent dans la suite de ce cours pour simplifier les choses), il y a une légère différence entre les 2 :

la référence est une référence implicite vers une donnée. Ainsi, dans un langage où l'on gère les entiers par référence, on pourra écrire `Integer i = 4`. On déclare `i` de type entier, même si la valeur contient en fait l'adresse de l'entier en mémoire

le pointeur est une référence explicite. Dans un langage où l'on manipule des pointeurs, on pourra écrire : `int i = 4 ; int* j = &i ;`. Ici, `i` est une variable de type entier qui contient la valeur 4 ; `j` est une variable de type « pointeur sur entier » qui contient l'adresse d'un entier (l'adresse de `i` en l'occurrence).

IV.3. Signification d'un nom de variable

Un nom de variable `n` cible d'une affectation représente la « case » de nom `n`.

Un nom de variable `n` à droite d'une affectation représente la valeur contenue dans la case de nom `n` ou référencée par la case de nom `n`.

Dans certains langages, on utilise une notation différente pour désigner la case et la valeur. C'est notamment le cas en shell : `VAR` représente la case de nom `VAR`, `$VAR` représente la valeur de la variable `VAR`.

IV.4. Exemples d'utilisations de variables

IV.4.1. En Python

Il n'est pas nécessaire de déclarer les variables. Par ailleurs, les variables n'ont pas de type. Les données sont stockées par référence.

```
a = 4
print(a)
a = [1, 2]
b = a
a.append(3)
print(a)
print(b)
```

L'extrait de code précédent affiche :

```
4
[1, 2, 3]
[1, 2, 3]
```

La variable `a` a pu contenir un entier, puis une liste.

En copiant le contenu de `a` dans `b` (ligne 4), on a en fait copié une référence. Ainsi la modification appliquée sur l'objet référencé par `a` (ligne 5) modifie également l'objet référencé par `b`... puisqu'il s'agit de la même donnée.

IV.4.2. En C

Les variables sont typées et doivent être déclarées. On manipule soit des valeurs soit des pointeurs. Une variable contenant un pointeur sur un type `t` est de type `t*`. On récupère le pointeur sur une donnée avec l'opérateur `&`. On récupère la donnée située à une adresse donnée avec l'opérateur `*`.

```
int c ;
int d ;
int *ptr1 ;
int *ptr2 ;
c = 4 ;
d = c ;
ptr1 = &c ;
ptr2 = ptr1 ;
c = 5 ;
printf("%d\n", c) ;
printf("%d\n", d) ;
printf("%d\n", *ptr1) ;
printf("%d\n", *ptr2) ;
```

Le programme précédent affiche :

```
5
4
5
5
```

En effet :

- À la ligne 6, on a copié la valeur de c (4) dans d. d contient maintenant 4.
- À la ligne 7, on stocke dans ptr1 l'adresse de c.
- À la ligne 8, on copie dans ptr2 le contenu de ptr1, à savoir l'adresse de c.
- À la ligne 9, on modifie la valeur de c, qui vaut maintenant 5.

Ainsi :

- c contient maintenant 5
- d contient toujours 4
- ptr1 contient l'adresse de c, \$donc à cette adresse, on trouve le contenu de c, soit 5
- ptr1 contient également l'adresse de c, donc à cette adresse, on trouve le contenu de c, soit 5

IV.4.3. En shell

Les variables ne sont pas déclarées et n'ont pas de type. Les données sont stockées par valeur. Pour récupérer la valeur stockée dans une variable, il faut la préfixer par le caractère \$.

```
a=3
b=a
c=$a
a=4
echo $a
echo $b
echo $c
```

Le programme précédent affiche :

```
4
a
3
```

En effet :

- à la ligne 2, on n'a pas copié la valeur de a dans b, mais on a stocké le texte « a » dans b
- à la ligne 3, on a copié le contenu de a (3) dans b
- à la ligne 4, on a modifié la valeur de a.

IV.4.4. En Pascal

Les variables sont typées et déclarées préalablement. De base, elles contiennent des valeurs.

```
var
    a, b : integer ;
begin
    a := 3 ;
    b := a ;
    a := 4 ;
    writeln(a) ;
    writeln(b) ;
end
```

Le programme précédent affiche :

```
4
3
```

On notera l'utilisation de la syntaxe «:= » pour l'affectation.

IV.4.5. En Java

Les variables doivent être déclarées et typées. Les « types simples » (nombres, caractères, booléen) sont stockés par valeur, les types « objet » et les tableaux sont stockés par référence.

```
int a, b ;
int [] t1 = {1, 2, 3} ;
int[] t2 ;
a = 2 ;
b = a ;
a = 3 ;
t1[0] = 4 ;
System.out.println(a) ;
System.out.println(b) ;
System.out.println(Arrays.toString(t1)) ;
System.out.println(Arrays.toString(t2)) ;
```

Le programme précédent affiche :

```
3
2
[4, 2, 3]
[4, 2, 3]
```

IV.5. Notion d'égalité

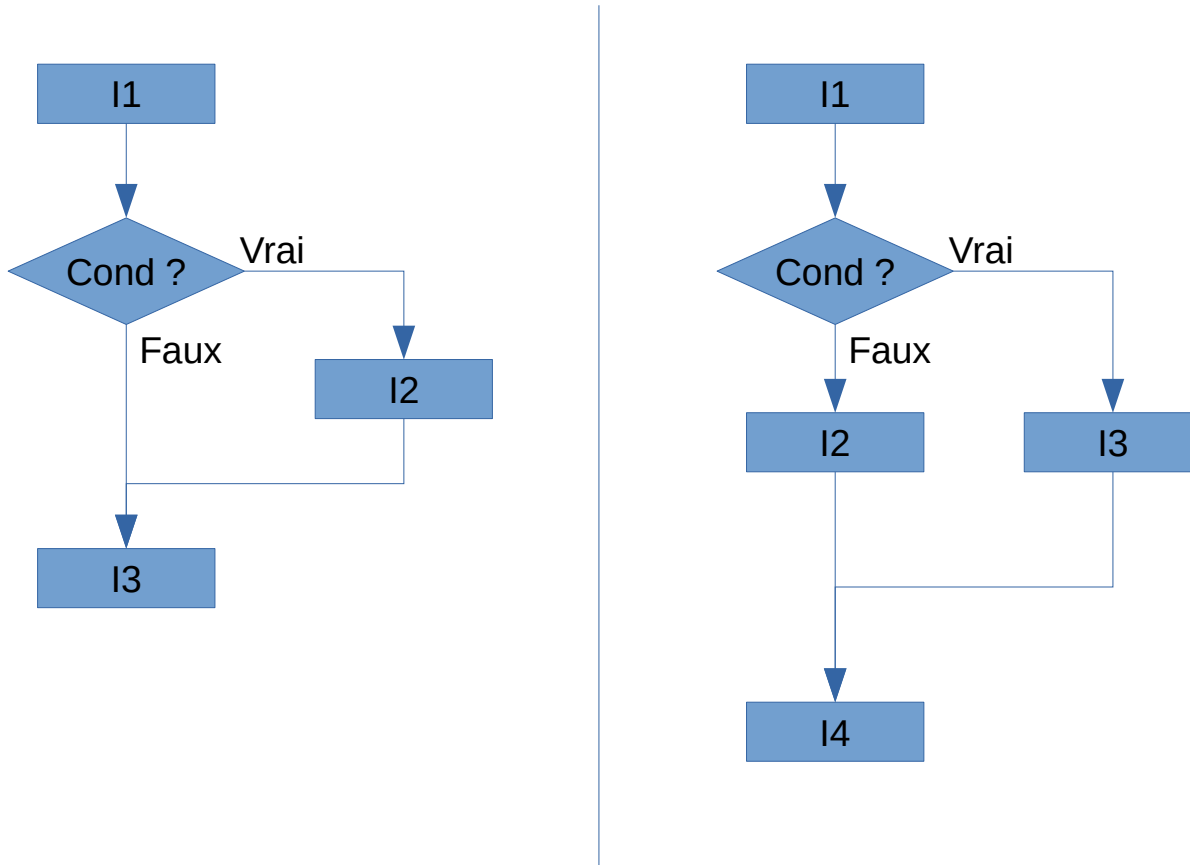
Dans beaucoup de langages, il existe 2 notions d'égalité, avec 2 notations différentes :

- un opérateur permettant de comparer les références (`is` en Python, `==` en Java par exemple)
- un opérateur permettant de comparer les valeurs (`==` en Python, `.equals()` en Java)

V. Les conditionnelles

V.1. Le test à 2 branches

Les langages impératifs définissent à peu près tous de **structures conditionnelles** correspondant aux schémas suivants :



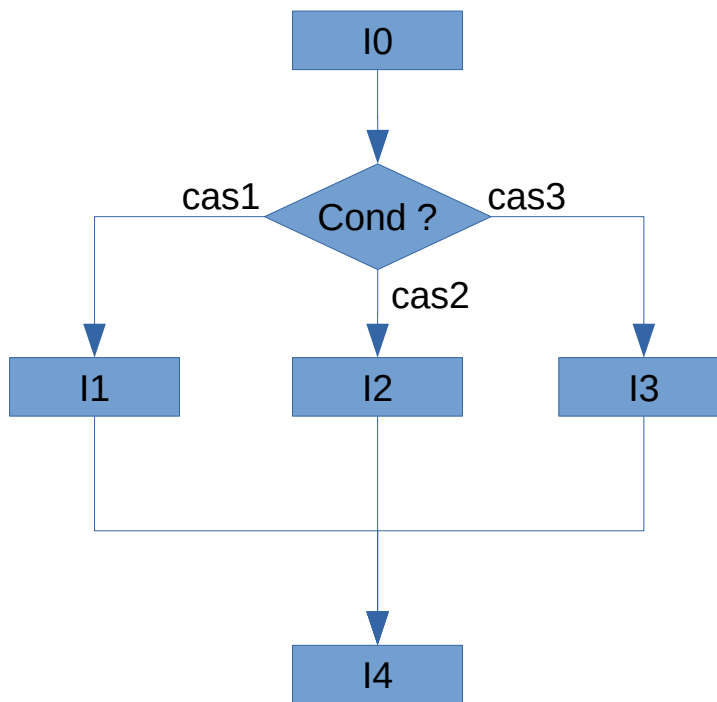
Le Schéma de gauche correspond à une structure *Si ... Alors ...* et le schéma de droite à une structure *Si ... Alors ... Sinon ...*

La partie suivant le « Si » (Cond? sur les schémas) doit être une *expression à valeur booléenne*. Ce qui suit le « Alors » et le « Sinon » doit être un bloc d'instructions.

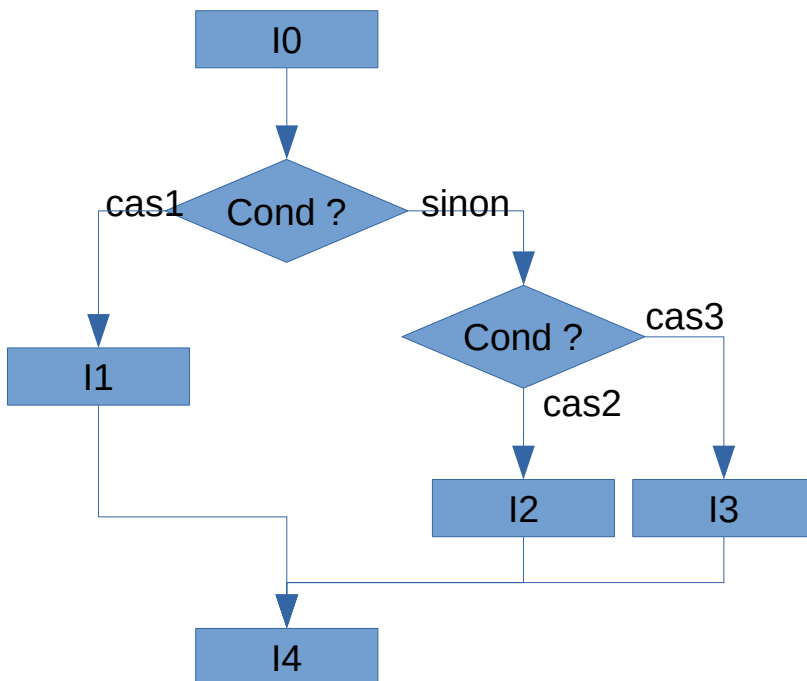
V.2. Le test à plus de 2 branches

Dans la plupart des langages de programmation, la conditionnelle si-alors-sinon ne propose que 2 branches, selon que la condition de teste est vraie ou fausse. Lorsque conceptuellement du résultat d'un test partent plus de 2 branches (la condition n'est alors plus à valeur booléenne), il est possible d'imbriquer des tests.

Ainsi, on traduit le schéma suivant :



En celui-là :



On note qu'il y a alors une séquentialité dans l'ordre d'évaluation des conditions.

Dans un langage comme Java ou C, cela peut s'écrire ainsi :

```
i0;
if (cas1) {
    i1;
} else {
    if (cas2) {
        i2;
    }
    else {
        i3;
    }
}
i4;
```

En Python, cela pourrait donner la structure suivante :

```
i0
if cas1:
    i1
else:
    if cas2:
        i2
    else:
        i3
i4
```

La multiplication des niveaux d'indentation rendant le code assez lourd, il existe une instruction spécifique en Python, `elif` qui permet d'implanter un `if` à plus de 2 branches :

```
i0
if cas1:
    i1
elif cas2:
    i2
else:
    i3
i4
```

Dans les langages avec un syntaxe dérivée du C, les accolades ne sont pas obligatoires lorsqu'un « else » ne contient qu'une instruction (pouvant être elle-même un if...). On peut donc procéder d'une manière similaire à ce que l'on peut faire en Python sans mot-clef spécifique :

```
i0;
if (cas1) {
    i1;
} else if (cas2) {
    i2;
}
else {
    i3;
}
i4;
```

Dans beaucoup de langages, il existe une instruction particulière, `switch`, qui permet d'exécuter un test à plus de 2 branches, mais uniquement lorsque le choix d'une branche correspond à une valeur précise d'une variable ($a = 1$, par exemple, mais pas $a < 2$).

Exemple en Java :

```
switch(a) {
    cas 1 -> System.out.println("a vaut 1");
    case 2-> System.out.println("a vaut 2");
    default-> System.out.println("a ne vaut ni 1 ni 2");
}
```

N.B. : en Python, un `match`, un peu similaire au `switch`, a été introduit dans la version 3.10.

V.3. Le test à valeur d'expression

Dans beaucoup de langages, il est possible de définir une expression dont la valeur dépend d'une condition. Cela s'effectue au moyen de ce qu'on appelle l'**opérateur ternaire**.

En C par exemple, cela s'écrit ainsi :

```
a = (b > 2) ? 4 : 3 ;
```

Cela signifie que Si **b** est supérieur à 2, c'est la valeur 4 qui est affectée à a. Sinon, c'est la valeur 3.

L'opérateur ternaire existe également en Python, mais avec une syntaxe différente. Ainsi, l'exemple précédent s'écrit ainsi en Python :

```
a = 4 if b > 2 else 3
```

L'opérateur ternaire définissant une expression, il doit donner une valeur dans tous les cas, ce qui rend le **else obligatoire**.

Exemple :

```
a = if (b > 2) then 4 else 3
```

VI. Les boucles

VI.1. Boucles bornées

La boucle bornée est une boucle qui permet d'effectuer un traitement (un bloc d'instructions) un nombre déterminé de fois.

VI.1.1. Boucle simple

La boucle simple est une boucle permettant de répéter un bloc d'instructions un nombre donné de fois.

Exemple en Logo :

```
repeat 4 [fd 100 rt 90]
```

Ce programme consiste à répéter 4 fois le bloc d'instructions [fd 100 rt 90].

VI.1.2. Boucles *for*

La boucle **for** est une boucle qui introduit un **compteur de boucle** et qui va définir un nombre de répétitions de son **corps** à partir des valeurs de ce compteur. En BASIC, par exemple cela s'écrit ainsi :

```
10 FOR I = 1 TO 10  
20 PRINT I  
30 NEXT I
```

Cela signifie que la ligne située entre le `FOR` et le `NEXT` (le corps de la boucle) va être exécutée 10 fois, la variable `I`, appelée variable de boucle, ou compteur, prenant successivement les valeurs de 1 à 10.

Dans beaucoup de langages, les boucles *for* ont la structure suivante :

```
for (i = 1; i <= 10; i = i + 1) {  
    i1;  
}
```

L'instruction `for` est suivie d'un triplet dont la signification de chacun des membres est la suivante :

- **premier membre** (`i = 1`) : initialisation avant la boucle ; exécutée une seule fois, avant la première itération ;
- **deuxième membre** (`i <= 10`) : condition de bouclage : avant d'exécuter le corps de la boucle, on vérifie si la condition de bouclage est vraie. Si c'est le cas, on exécute le corps de la boucle, sinon, on passe à l'instruction suivante.
- **Troisième membre** (`i = i + 1`) : mise à jour du compteur de boucle. Effectuée à la fin de chaque exécution du corps de boucle.

Attention : même si cela est souvent autorisé par les langages, il n'est *a priori* pas recommandé de modifier la valeur d'un compteur de boucle dans le corps d'une boucle *for*.

VI.1.3. Boucles *for each*

Une boucle **for each** est une boucle bornée qui permet de parcourir les éléments d'une collection de données. Si dans certains langages, ces boucles sont introduites par le mot-clé `foreach` (c'est le cas par exemple en *Perl*), dans beaucoup d'autres langages, c'est le mot-clé `for` qui les introduit (mais avec une syntaxe différente de celle utilisée pour les boucles *for* vues précédemment).

En Python, une boucle *for each* permet par exemple de parcourir facilement les éléments d'une liste lorsque les indices ne sont pas nécessaires :

```
liste = [1, 2, 3, 4]
total = 0
for valeur in liste:
    total += valeur
print(total)
```

En Java, on pourrait écrire cela ainsi :

```
List<Integer> liste = List.of(1, 2, 3, 4);
int total = 0;
for (int v: liste) {
    total += v
}
System.out.println(total);
```

Pour pouvoir utiliser une boucle *for each*, il faut que la structure à parcourir soit *itérable*. Une **structure itérable** est une structure dont il est possible de parcourir l'ensemble des éléments un à un, grâce à un mécanisme appelé *itérateur*. Un itérateur est un objet associé à une structure de données itérable proposant essentiellement 2 méthodes `next()` et `isDone()`. `next()` permet de passer à l'élément suivant de la structure, et `isDone()` permet de savoir si on a parcouru tous les éléments de la structure.

Les boucles *for each* permettent également de parcourir des structures dont les éléments ne sont calculés qu'à la demande (via l'utilisation de la méthode `next()` de l'itérateur associé). On appelle de telles structures des **structures paresseuses**. La structure `range` de Python, construite grâce à une instruction comme `range(10)`, constitue un exemple d'une telle structure. Ainsi, en Python, il n'existe pas vraiment de boucle *for*, mais on utilise une boucle *for each* pour implanter la notion de boucle *for* :

```
for i in range(1, 11) :
    print(i)
```

VI.2. Boucles non bornées

Les boucles non bornées sont des boucles dont le nombre de répétitions n'est *a priori* pas connu à l'avance. Le corps de la boucle doit être répété tant qu'une condition est vraie, ou jusqu'à ce qu'une condition soit fausse. La version la plus utilisée des boucles non bornées est la boucle *while*¹.

La structure d'une boucle *while* est à peu près standard. En Python, on écrit ainsi une boucle *while* :

```
nb = int(input())
while (nb < 0 or nb > 9) :
    print("valeur incorrecte")
    nb = int(input())
```

Ainsi, le corps de la boucle sera répété tant que *nb* n'est pas dans l'intervalle [0, 9].

Le test qui suit le mot-clé *while* est évalué avant chaque exécution du corps de la boucle, y compris la première. Cela signifie qu'**il est tout à fait possible de ne jamais exécuter le corps d'une boucle *while***. C'est le cas dans l'exemple ci-dessus si le premier nombre rentré par l'utilisateur est directement valide. La non prise en compte de cette possibilité peut être à l'origine de bugs.

Il est important de s'assurer qu'une boucle *while* termine. Là encore, la non terminaison d'une boucle peut être à l'origine de bugs (dans l'exemple ci-dessus, cela nécessite de supposer que l'utilisateur va finir par rentrer un nombre valide. Dans l'absolu, il faudrait autoriser un nombre maximum d'essais et soit interrompre le programme proprement, soit décider d'une valeur par défaut si ce nombre maximum est atteint).

Il existe une autre forme de boucle non bornée : la boucle *do ... while*. Dans cette version le test est exécuté en fin de boucle. On est donc sûr de rentrer au moins une fois dans le corps de la boucle.

Malgré les noms, une boucle bornée peut s'exécuter un nombre infini de fois, et une boucle non bornée peut être exécutée un nombre fixe de fois.

VI.3. On a fait le tour ?

Les types de boucle vus ici correspondent aux types les plus fréquemment rencontrés. On peut cependant trouver des versions plus ou moins différentes dans d'autres langages de programmation (boucle *repeat ... until* en Pascal, boucles *from ... until ... loop... end* d'Eiffel, etc.).

¹ Il existe aussi une boucle *do ... until ...*

VII. Les sauts

VII.1. saut « sans retour »

Dans certains langages (comme le basic), et notamment dans les langages de bas niveau (les langages machines), il est possible d'utiliser une instruction « goto » qui permet d'aller directement à une autre instruction (située avant ou après l'instruction courante).

Exemple en Basic AppleSoft :

```
10 PRINT "bonjour"  
20 GOTO 200  
50 PRINT "monde"  
80 END  
150 PRINT "le"  
180 GOTO 50  
200 PRINT "tout"  
250 GOTO 150
```

Ce programme affiche « Bonjour tout le monde ».

Il est cependant évident que ce code est peu lisible.

VII.2. Saut « avec retour »

Dans certains langages (comme le basic), et notamment dans les langages de bas niveau (les langages machines), il est possible d'utiliser une instruction « gosub » qui permet d'aller directement à une autre instruction (située avant ou après l'instruction courante). À partir de ce moment-là, quand une instruction return sera rencontrée, l'exécution reprendra à l'instruction suivant l'instruction gosub.

Exemple en Basic :

```
10 GOSUB 200  
20 PRINT "Hello"  
30 GOSUB 200  
40 PRINT "World"  
50 GOSUB 200  
100 END  
200 PRINT "*"   
220 RETURN
```

Ce programme affiche « *Hello*World* ».

VII.3. Remarque générale concernant les sauts

Les sauts ont petit à petit disparu des langages de programmation car ils rendent le programme particulièrement difficile à comprendre. Ainsi, dans les langages « modernes » qui les proposent encore, leur utilisation reste forcément déconseillée.

VII.4. Sauts spécifiques aux boucles

Dans les langages modernes, des instructions assimilables à des sauts et rattachées aux boucles perdurent. Les plus classiques sont les suivantes :

- **break** : permet de sortir prématurément d'une boucle
- **continue** : permet de sortir prématurément de l'itération courante de la boucle et de continuer au début de l'itération suivante.

L'usage intensif de ces instructions est déconseillé pour les mêmes raisons que pour les sauts génériques.

VIII. Les fonctions

VIII.1. Éviter la redondance de blocs d'instructions

On parle de **redondance** dans un code lorsqu'on trouve plusieurs fois la même séquence d'instruction, ou encore la même expression.

Exemple en Python :

```
def prog():
    for i in range(40):
        print("-", end='')
    print()
    print("Hello")
    for i in range(40):
        print("-", end='')
    print()
    quitter = False
    while not quitter:
        print("1. Afficher un message de bienvenue")
        print("2. Quitter")
        reponse = input()
        if reponse == "2":
            quitter = True
        elif reponse == "1":
            print("Bonjour tout le monde")
    for i in range(40):
        print("-", end='')
    print()
```

La redondance présente 2 inconvénients

- perte de place (inconvenient très peu important de nos jours) ;
- **risque d'incohérence**

À retenir :

redondance → risque d'incohérence

Risque d'incohérence : si on souhaite changer quelque chose dans le code (par exemple parce qu'il est buggué) et si ce qui est à changer est dans une partie répétée, on risque d'oublier d'effectuer le changement à certains endroits

On peut donc **factoriser des suites d'instructions** en introduisant des **procédures**² (ou fonctions à valeur de procédures) :

```
def affiche_ligne_tirets(): # définition de la procédure
    for i in range(40):
        print("-", end='')
    print()

def prog():
    affiche_ligne_tirets() # appel de la procédure
    print("Hello")
    affiche_ligne_tirets()
    quitter = False
    while not quitter:
        print("1. Afficher un message de bienvenue")
        print("2. Quitter")
        reponse = input()
        if reponse == "2":
            quitter = True
        elif reponse == "1":
            print("Bonjour tout le monde")
    affiche_ligne_tirets()
```

Autre avantage des procédures : en nommant un bloc d'instructions, on le documente. Ceci permet aussi d'éviter d'écrire des codes trop longs (une « fonction » devrait normalement tenir sur un écran → ±24 lignes max de 80 caractères max).

Les procédures peuvent également être **paramétrées**.

Les paramètres peuvent être vus comme des variables locales à la fonction dont la valeur initiale est fixée lors de l'appel. On distingue les **paramètres formels** (le nom donné dans la définition de la fonction) des **paramètres effectifs** (la valeur passée lors de l'appel).

² Dans beaucoup de langages, il n'y a pas vraiment de différence syntaxique entre fonction et procédure, même si la différence sémantique demeure

Exemple :

```
def affiche_ligne_tirets(symbole, nb_symboles):
    # symbole et nb_symboles sont des paramètres formels
    for i in range(nb_symboles):
        print(symbole, end='')
    print()

def affiche_gros_separateur():
    affiche_ligne_tirets("=", 80)
    # "=" et 80 sont des paramètres effectifs

def affiche_petit_separateur():
    affiche_ligne_tirets("-", 40)

def affiche_entete():
    affiche_gros_separateur()
    print("Hello")
    affiche_petit_separateur()

def affiche_menu():
    print("1. Afficher un message de bienvenue")
    print("2. Quitter")
    affiche_petit_separateur()

def prog():
    affiche_entete()
    quitter = False
    while not quitter:
        affiche_menu()
        reponse = input()
        if reponse == "2":
            quitter = True
        elif reponse == "1":
            print("Bonjour tout le monde")
    affiche_gros_separateur()
```

Il est possible de « sortir » prématurément d'une procédure en utilisant l'instruction `return`. Ceci est cependant souvent déconseillé car cela complique la compréhension du code en modifiant le flux d'exécution

VIII.2. Éviter la redondance d'expressions

Tout comme il est possible d'éviter la redondance d'une suite d'instructions, il est possible d'éviter la redondance d'une expression aux moyens de **fonctions**. Contrairement aux procédures, l'exécution d'une fonction fournit une valeur. Ainsi, l'appel d'une fonction est une forme d'expression. Aussi, dans les fonctions, l'instruction `return` est obligatoire et doit être suivie d'une expression qui représentera la valeur de l'expression que forme l'appel de la fonction. On appelle cette valeur la **valeur de retour** de la fonction.

Exemple sans fonction :

```
from math import sqrt

def prog():
    x1 = 1
    y1 = 3
    x2 = 2
    y2 = 4
    x3 = 5
    y3 = 1

    d12 = sqrt((x1 - x2)** 2 + (y1 - y2)**2)
    d13 = sqrt((x1 - x3)** 2 + (y1 - y3)**2)
    d23 = sqrt((x3 - x2)** 2 + (y3 - y2)**2)

    if d12 < d13:
        min1 = d12
    else:
        min1 = d13
    if d23 < min1:
        min1 = d23

    print("distance minimum : ", min1)
```

Exemple avec fonctions :

```
from math import sqrt

def diff(b, a):
    return a - b

def diff_carree(a, b):
    return diff(a, b) ** 2

def dist(x1, y1, x2, y2):
    return sqrt(diff_carree(x1, x2) + diff_carree(y1, y2))

def prog():
    x1 = 1
    y1 = 3
    x2 = 2
    y2 = 4
    x3 = 5
    y3 = 1

    d12 = dist(x1, y1, x2, y2)
    d13 = dist(x1, y1, x3, y3)
    d23 = dist(x3, y3, x2, y2)

    if d12 < d13:
        min1 = d12
    else:
        min1 = d13
    if d23 < min1:
        min1 = d23

    print("distance minimum : ", min1)
```

Là encore, l'utilisation de plusieurs return est possible, mais n'est pas forcément conseillée :

```
def present(valeur, liste):
    for val in liste:
        if val == valeur:
            return True
    return False

liste = [2, 4, 1, 8, 3, 9]

print("Cas 1 : ", present(12, liste))
print("Cas 2 : ", present(4, liste))
```

VIII.3. Différences procédures/fonctions

La différence entre procédures et fonctions est essentiellement sémantique, rarement complètement syntaxique. On emploiera donc souvent le terme fonction pour désigner aussi bien les « vraies » fonctions que les procédures :

- dans certains langages comme Pascal, le mot-clé utilisé pour définir une procédure et une fonction n'est pas le même ;
- dans les langages pour lesquels il faut typer les fonctions, le type de retour des procédures est un type spécifique (souvent, `void`) spécifiant que cette fonction ne doit rien renvoyer ;
- dans la plupart des langages, on pourra distinguer les fonctions par le fait qu'il y a au moins une instruction `return` et que toutes les instructions `return` sont suivies d'une expression.

Il est en général possible d'utiliser un appel de fonction comme s'il s'agissait d'une instruction, c'est-à-dire d'un appel de procédure. La valeur de retour est alors tout simplement ignorée.

VIII.4. les méthodes

Dans les langages orientés objets, on utilise des **classes** plutôt que des enregistrements pour définir de nouveaux types. Dans une classe, on pourra définir des **méthodes**, qui jouent un rôle similaire aux fonctions et procédures, sauf qu'elles sont **appliquées** à une donnée (appelée plutôt **objet** ou **instance**) de la classe, accessible plus ou moins implicitement dans la méthode.

Par ailleurs, on peut également ajouter aux classes des constructeurs, sortes de méthodes chargées d'initialiser un objet. Par contre, un constructeur ne s'applique pas à un objet.

Exemple en java, version 1 :

```
public class Personne {
    private String nom;
    private String prenom;
    private int age;
    public Personne(String nomInitial, String prenomInitial,
                    int ageInitial) {
        // constructeur. En java, il a le même nom que la classe
        this.nom = nomInitial;
        this.prenom = prenomInitial;
        this.age = ageInitial;
    }

    public void affiche() {
        // this représente, de manière implicite,
        // la 'personne' SUR LAQUELLE ON APPELLE LA METHODE
        System.out.println(this.prenom + " " + this.nom);
    }

    public static void main(String... args) {
        Personne p1 = new Personne("Cadivel", "Martin", 21);
        Personne p2 = new Personne("Perret", "Gaële", 16);
        p1.affiche(); // une méthode est appliquée à un
objet
        p2.affiche();
    }
}
```

Exemple en java, version 2 :

```
public class Personne {
    private String nom;
    private String prenom;
    private int age;
    public Personne(String nomInitial, String prenomInitial,
                    int ageInitial) {
        // le mot clé "this" est... facultatif !
        nom = nomInitial;
        prenom = prenomInitial;
        age = ageInitial;
    }

    public void affiche() {
        // le mot clé "this" est... facultatif !
        System.out.println(prenom + " " + nom);
    }

    public static void main(String... args) {
        Personne p1 = new Personne("Cadivel", "Martin", 21);
        Personne p2 = new Personne("Perret", "Gaële", 16);
        p1.affiche(); // une méthode est appliquée à un
objet
        p2.affiche();
    }
}
```

La façon de déclarer les méthodes peut varier d'un langage à un autre. Ainsi, en Python, la mention dans une méthode de l'objet auquel est appliquée la méthode doit être explicite. Il s'agit du premier paramètre de la méthode, et il est nommé par convention `self`. Par ailleurs, le constructeur s'appelle `__init__`.

```

class Personne:
    def __init__(self, nomInitial, prenomInitial, ageInitial):
        # le constructeur
        self.nom = nomInitial
        self.prenom = prenomInitial
        self.age = ageInitial

    def affiche(self) :
        # le self est obligatoire
        print(self.prenom, self.nom)

p1 = Personne("Cadivel", "Martin", 21);
p2 = Personne("Perret", "Gaële", 16);

p1.affiche() # dans la méthode, self aura pour valeur p1
p2.affiche()

```

Certains langages orientés objets permettent de déclarer aussi bien des fonctions que des méthodes (javascript, python, kotlin par exemple), d'autres ne connaissent que les méthodes (Java, Eiffel).

IX. Interprétation vs Compilation

IX.1. Exécution d'un programme

L'exécution d'un programme est réalisée grâce à une **machine**. Dans certains cas, cette machine est directement le micro-processeur de l'ordinateur. Mais celui-ci ne connaissant que le langage machine, il faut donc :

- soit programmer directement en langage machine/assembleur ;
- soit écrire le programme dans un langage de plus haut niveau, (on parlera de **code source**) et le traduire une fois pour toute en assembleur (on parlera de **code objet**).

Écrire en assembleur est relativement laborieux car le langage, peu « évolué », ne propose quasiment aucune facilité. On utilise donc plus souvent la deuxième manière. Le programme qui effectue cette traduction est appelée **compilateur**.

Dans d'autres cas, la « machine » est en fait un programme qui exécute, ligne à ligne le code que l'on a écrit. Ce programme est appelé **interpréteur**.

Enfin, dans certains cas, la « machine » sera un interprète d'un langage intermédiaire. Il faut donc commencer par compiler le code source en **code intermédiaire** (souvent appelé aussi bytecode), avant d'interpréter ce code intermédiaire. On parle alors de langage **semi-compilé**.

IX.2. Exemple de mise en œuvre avec un programme de type « Hello World »

IX.2.1. Langage interprété : exemple en Python

Fichier source `hello.py` :

```
print("Hello World")
```

Exécution :

```
python3 hello.py
```

python3 est l'interprète ; il analyse et exécute directement le fichier source

IX.2.2. Langage compilé : exemple en C

Fichier source `hello.c` :

```
#include<stdio.h>
int main() {
    printf("Hello World\n") ;
}
```

Compilation :

```
gcc -o hello hello.c
```

gcc est le compilateur. Il crée le fichier compilé (« hello ») en langage machine

Exécution :

```
./hello
```

On exécute directement le fichier compilé.

IX.2.3. Langage semi-compilé : exemple en Java

Fichier source `Hello.java` :

```
public class Hello {
    public static void main(String... args) {
        System.out.println("Hello World") ;
    }
}
```

Compilation :

```
javac Hello.java
```

javac est le compilateur. Il crée un fichier semi-compilé (`Hello.class`) à partir du fichier source.

Exécution :

```
java Hello
```

java est l'interpréteur de bytecode. Il interprète le code semi-compilé contenu dans le fichier `Hello.class`.

IX.3. Interprétation vs Exécution : avantages et inconvénients

Un langage interprété est souvent plus simple à mettre au point : dès que le programme est modifiée, une nouvelle exécution prendra en compte les modifications. Un langage compilé, comme un langage semi-compilé, nécessite par contre toujours de passer par la phase de recompilation, processus pouvant être long sur un gros programme.

Un programme écrit dans un langage interprété est souvent portable (utilisable sur des ordinateurs et systèmes différents) : il suffit que l'interprète du langage (la « machine ») existe dans l'environnement cible, alors qu'un programme écrit en assembleur est en général dédié à un système d'exploitation sur une architecture donnée.

Comme ce qui est exécuté avec un langage interprété, c'est le « code source », cela rend en général transparent l'algorithme d'un programme (à moins d'utiliser des techniques d'*obfuscation* (obscurcissement) du code, ce qui n'est pas toujours souhaitable (secret industriel...)).

L'exécution d'un langage compilé est en général beaucoup plus rapide que celle d'un langage interprété, puisque c'est le micro-processeur qui exécute directement le « code objet ».

Dans le cas des langages semi-compilés, le code intermédiaire est souvent tellement proche du langage machine que son exécution est, en termes de rapidité, bien plus proche d'un langage compilé que d'un langage interprété. Par ailleurs, le code intermédiaire est générique, ce qui le rend portable, tout en cachant relativement bien l'implantation de l'algorithme.

Quelques exemples de langage :

- Langages compilés
 - C
 - Pascal
 - Rust
- Langages interprétés
 - Perl
 - Javascript
 - Python
- Langages semi-compilés
 - Java

IX.4. Autres situations

Certains « interpréteurs » utilisent de la compilation « juste à temps (*Just In Time*, ou JIT) : cela consiste à compiler « à la volée » les parties d'un programme les plus gourmandes en temps de calcul. On a alors les caractéristiques d'un langage interprété, mais en réduisant énormément l'inconvénient « lenteur ».

Le langage Python est un langage interprété, mais :

- dans certains cas, il compile « à la volée » le code source dans un code intermédiaire, mais non portable ;
- beaucoup de « bibliothèques » Python sont en fait écrites en C (donc compilées), et il existe juste une possibilité d'appeler les fonctions qu'elles définissent depuis un programme Python comme s'il s'agissait de fonctions Python. On bénéficie alors d'une vitesse d'exécution proche de celle d'un langage compilé.

IX.5. Facilités de mise en œuvre

Pour les langages interprétés, il est souvent possible, dans les systèmes unique, de faire commencer le code par une ligne `#!/chemin` d'accès à l'interpréteur. En donnant les droits d'exécution au fichier source, on peut alors l'exécuter directement comme s'il s'agissait d'un programme compilé (on passe bel et bien par l'interpréteur, mais c'est le système d'exploitation qui fait cet appel).

Pour les langages semi-compilés, l'appel au compilateur peut également être fait automatiquement par l'interpréteur si on spécifie directement un fichier source. Le fichier semi-compilé est créé temporairement, et supprimé sitôt l'exécution du programme terminée.

IX.5.1. Exemple en Python

Fichier `hello.py`

```
#!/usr/bin/python3
print("Hello World")
```

Attribution des droits d'exécution sur le fichier source :

```
chmod ugo+x hello.py
```

Exécution :

```
./hello.py
```

IX.5.2. Exemple en Java

Fichier source `Hello.java` :

```
public class Hello {
    public static void main(String... args) {
        System.out.println("Hello World") ;
    }
}
```

Exécution :

```
java Hello.java
```

`java`, l'interpréteur appelle implicitement le compilateur. Puis exécute le fichier compilé.

Depuis Java 25, il n'est même plus indispensable, dans de ce cas-là, de définir une classe. On peut écrire le fichier `Hello.java` suivant :

```
void main() {
    IO.println("Hello World") ;
}
```

puis l'exécuter ainsi :

```
java Hello.java
```

X. Niveau de langage

Plus un langage est proche de l'assembleur, plus on parlera de langage de « bas niveau ». Plus un langage ajoute de concepts et structures, plus on parlera de langage de « haut niveau ».

Le niveau le plus bas est le langage machine. On trouvera ensuite des langages comme le C, par exemple, puis le Pascal, qui sont des langages qui nécessitent de connaître beaucoup la représentation des données en machine. Des langages comme Python, Java ou Processing seront par contre des langages de haut niveau (même si, pour bien comprendre leur fonctionnement, il faut avoir quand même une idée de la façon dont les données sont représentées).

Enfin, on parlera de langage de quatrième génération (L4G) quand on parle d'un langage fait pour décrire des fonctionnements automatisés d'une application (Visual Basic For Application, par exemple).